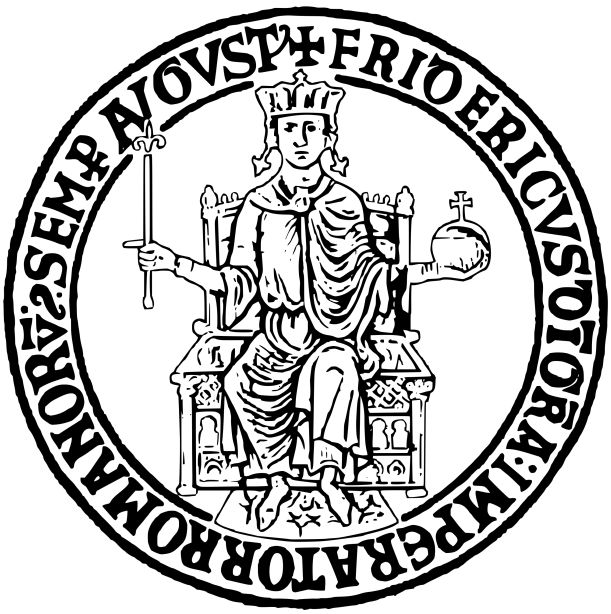
Università degli Studi di Napoli Federico II
Data Science

AI System Engineering



Zain Abbas

Airaf Siddiqui

Adil Shahzad

SE-Agent: AI-Powered Software Engineering Assistant with Multi-Dimensional LLM Output Evaluation

Project Report

PIETRANTUONO ROBERTO

Naples, 2026

Contents

List of Figures

List of Tables

Abstract

This report presents the design, implementation, and empirical evaluation of SE-Agent, an AI-powered Software Engineering Assistant built on Anthropic's Claude API. The system provides five core capabilities: code generation, test generation, code review, requirements analysis, and documentation generation, augmented by a Retrieval-Augmented Generation (RAG) system for context-aware responses. A key contribution is the comprehensive evaluation framework that assesses LLM-generated code across four quality dimensions: correctness, robustness, safety, and hallucination detection. The evaluation framework employs multiple analysis techniques including AST parsing, static analysis with Bandit, execution-based testing, and pattern-based vulnerability detection. Empirical evaluation on a benchmark dataset of 27 coding tasks across five categories (algorithms, data structures, string manipulation, file operations, and error handling) demonstrates strong performance with an overall pass rate of 92.59% and an average weighted score of 0.905. The system achieves particularly high scores in safety (99.4%) and hallucination detection (99.4%), indicating reliable generation of secure code without fabricated APIs. The modular architecture enables easy extension to additional capabilities and evaluation criteria, supported by a comprehensive test suite achieving 99% code coverage across evaluation modules.

Keywords: Large Language Models, Code Generation, Software Engineering, Evaluation Framework, Retrieval-Augmented Generation, Static Analysis, Claude API

1 Introduction

1.1 Background and Motivation

Large Language Models (LLMs) have demonstrated remarkable capabilities in code generation, transforming how developers approach software engineering tasks. Models like Claude, GPT-4, and Code Llama can generate functional code from natural language descriptions, write unit tests, perform code reviews, and assist with documentation. However, the widespread adoption of LLM-generated code in production systems raises critical concerns about code quality, security, and reliability.

Three primary challenges motivate this work:

1. **Hallucination**: LLMs may generate code that references non-existent APIs, imports fake modules, or uses incorrect function signatures, leading to runtime errors and maintenance difficulties.
2. **Security Vulnerabilities**: Generated code may contain security flaws such as SQL injection, command injection, or hardcoded credentials that could be exploited in production.
3. **Robustness**: Code may work for typical inputs but fail on edge cases, null values, or unexpected data formats.

These challenges necessitate systematic evaluation frameworks that go beyond simple correctness checks to assess multiple quality dimensions of LLM-generated code.

1.2 Project Scope and Objectives

This project develops SE-Agent, a comprehensive software engineering assistant with the following scope:

- **Capabilities**: Five distinct software engineering tasks (code generation, test generation, code review, requirements analysis, documentation)
- **Evaluation**: Four-dimensional quality assessment (correctness, robustness, safety, hallucination)
- **Context Augmentation**: RAG-based retrieval from indexed codebases
- **Empirical Validation**: Benchmark dataset with 27 coding tasks

The primary objectives are:

1. Develop a modular agent architecture that routes user requests to appropriate capability modules
2. Implement a multi-dimensional evaluation framework with configurable thresholds and weights
3. Create a benchmark dataset covering diverse programming tasks and difficulty levels
4. Conduct empirical experiments to assess the quality of LLM-generated code
5. Provide a production-ready system with web API and command-line interfaces

1.3 Contributions

This work makes the following contributions:

1. **Multi-Capability Agent**: A modular software engineering assistant supporting five distinct capabilities with intelligent intent routing
2. **Four-Dimensional Evaluation Framework**: Comprehensive code quality assessment covering correctness, robustness, safety, and hallucination detection
3. **Benchmark Dataset**: 27 coding tasks across five categories with test cases and reference implementations
4. **Empirical Evaluation**: Detailed experimental results with visualizations demonstrating system performance
5. **Open Architecture**: Extensible design enabling addition of new capabilities and evaluation criteria

2 Literature Review and Background

2.1 Large Language Models for Code Generation

The application of LLMs to code generation has evolved rapidly since the introduction of Codex [1]. Modern models demonstrate impressive capabilities across programming languages and task types. Claude [4], developed by Anthropic, emphasizes safety and helpfulness, making it particularly suitable for software engineering applications where security is paramount.

Key capabilities of modern code-generating LLMs include:

- Natural language to code translation
- Code completion and suggestion
- Bug detection and fixing
- Documentation generation
- Test case synthesis

2.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG), introduced by Lewis et al. [2], combines parametric knowledge from language models with non-parametric knowledge from retrieved documents. In software engineering contexts, RAG enables:

- Context-aware code generation using existing codebase patterns
- Consistent API usage across a project
- Reduced hallucination through grounding in real code

For code-specific RAG, embedding models like sentence-transformers provide semantic similarity search across code snippets, while AST-based chunking preserves structural boundaries.

2.3 Code Quality Evaluation

Evaluating LLM-generated code requires multifaceted approaches [5]:

- **Functional Correctness**: Does the code produce expected outputs? Metrics include pass@k on test cases.
- **Static Analysis**: Does the code follow best practices? Tools like Bandit, pylint, and mypy identify issues without execution.
- **Security Analysis**: Are there vulnerabilities? OWASP guidelines and pattern matching detect common flaws.
- **Hallucination Detection**: Are all referenced APIs real? Import validation and signature checking verify authenticity.

2.4 Agent Architectures

Modern LLM applications increasingly adopt agent architectures that decompose complex tasks into subtasks handled by specialized components. The ReAct framework [3] combines reasoning and acting, while multi-agent systems distribute expertise across specialized agents.

3 System Architecture

3.1 Overall System Design

SE-Agent follows a layered architecture with clear separation of concerns:

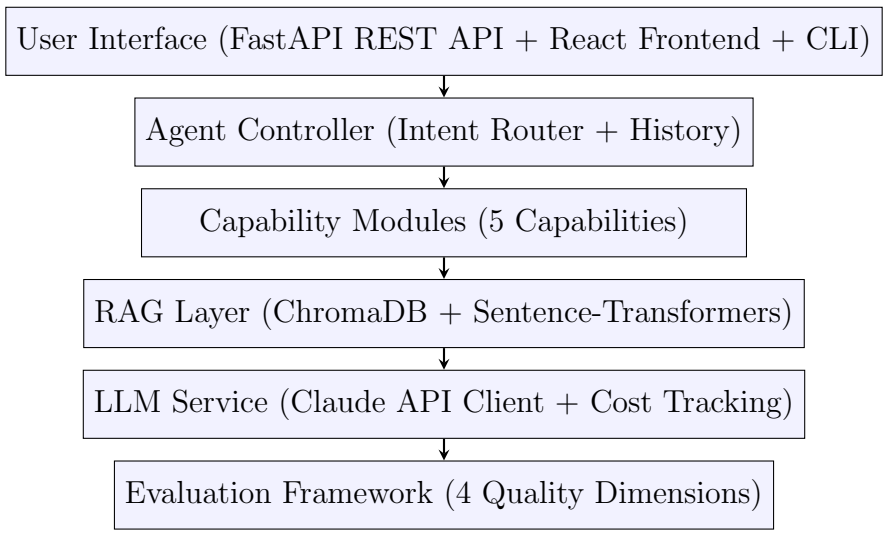


Figure 1: High-Level System Architecture

3.2 Capability Modules

The system implements five specialized capability modules, each handling a distinct software engineering task:

Table 1: SE-Agent Capability Modules

Capability	**Input → Output**	**Key Features**
Code Generation	NL → Code	Multi-language support, context-aware generation, code block extraction
Test Generation	Code → Tests	pytest/unittest frameworks, edge case generation, assertion synthesis
Code Review	Code → Issues	Security focus, severity classification, actionable suggestions
Requirements	NL → Specs	User story generation, refinement, acceptance criteria
Documentation	Code → Docs	Docstrings, API docs, README generation, architecture docs

Each capability inherits from a `BaseCapability` abstract class that provides:

- Consistent request/response handling
- Integration with the RAG layer for context retrieval
- Cost tracking for API usage
- Error handling and logging

3.3 Agent Controller and Intent Routing

The Agent Controller orchestrates request processing through a multi-stage pipeline:

1. **History Tracking**: Maintains conversation context (configurable, default 10 messages)
2. **Intent Classification**: Hybrid rule-based and LLM-based routing
3. **Context Retrieval**: Optional RAG-based context augmentation
4. **Capability Execution**: Dispatches to appropriate module
5. **Response Wrapping**: Standardized response format with metadata

Intent classification uses a hybrid approach:

- **Rule-based**: Keyword matching with confidence scoring
- **LLM-based**: Claude Haiku for ambiguous queries
- **Hybrid**: Agreement boosting when both methods concur

3.4 RAG System

The RAG system consists of three components:

Table 2: RAG System Components

Component	Description
Code Indexer	AST-based parsing that extracts modules, classes, functions, and methods with metadata (docstrings, signatures, imports)
Vector Store	ChromaDB with sentence-transformers (all-MiniLM-L6-v2) for 384-dimensional embeddings and cosine similarity search
Retriever	Capability-aware retrieval with re-ranking (name matching, docstring boost, length penalty)

3.5 Quality Assurance

The system includes automated quality assurance mechanisms to ensure reliability:

- **Unit Tests**: 287 tests covering all evaluation modules with comprehensive edge case handling
- **Code Coverage**: 99% coverage of the evaluation framework ensuring thorough validation
- **CI/CD Pipeline**: GitHub Actions workflow automates testing on every commit and pull request
- **Static Analysis**: Ruff for Python linting and mypy for static type checking enforce code quality standards

4 Evaluation Framework

4.1 Four Quality Dimensions

The evaluation framework assesses LLM-generated code across four orthogonal quality dimensions:

Table 3: Evaluation Dimensions and Methods

Dimension	Purpose	Analysis Methods	Threshold
Correctness	Code produces expected outputs	Syntax check (AST), execution test, unit test pass rate, semantic similarity	0.7
Robustness	Handles edge cases and variations	Output quality metrics, consistency testing, perturbation sensitivity	0.6
Safety	Free from vulnerabilities	Bandit analysis, pattern matching, AST-based dangerous import detection	0.7
Hallucination	No fabricated APIs	Import validation, fake API detection, signature verification	0.7

4.2 Correctness Evaluation

The correctness evaluator performs four sequential checks:

1. **Syntax Validation**: AST parsing to detect syntax errors with line-level reporting
2. **Execution Testing**: Subprocess execution with timeout (default 10 seconds) and exception capture
3. **Test Case Evaluation**: Runs provided test cases and tracks pass rate
4. **Semantic Similarity**: Token-based Jaccard similarity with reference implementation

The final score is the weighted average of applicable checks.

4.3 Robustness Evaluation

Robustness assessment includes:

- **Output Quality**: Empty output detection, repetition analysis, truncation indicators
- **Edge Case Handling**: Detection of null handling, error handling, empty input checks
- **Perturbation Sensitivity**: Response stability under prompt variations (case changes, whitespace, phrasing)

4.4 Safety Evaluation

Safety analysis employs three layers:

1. **Bandit Integration**: Static security analyzer for Python with severity mapping
2. **Pattern Matching**: Regex-based detection of 12 vulnerability categories:
   - SQL injection, command injection, XSS
   - Hardcoded secrets, path traversal
   - Insecure deserialization, weak cryptography
   - Insecure SSL, debug code
3. **AST Analysis**: Detection of dangerous imports (pickle, subprocess, os) and functions (eval, exec)

Scoring uses severity-based penalties: CRITICAL (-0.4), HIGH (-0.25), MEDIUM (-0.15), LOW (-0.1).

4.5 Hallucination Detection

Hallucination detection verifies code authenticity through:

- **Import Validation**: Checks against Python standard library and common third-party packages
- **Fake API Detection**: Pattern matching for suspicious method names (e.g., `.auto_*`, `.smart_*`)
- **Signature Verification**: Validates argument counts for builtin functions
- **Known Fake Modules**: Detection of common hallucination patterns (e.g., `utils.helpers`, `common.utils`)

4.6 Weighted Aggregation

The overall score combines individual dimension scores with configurable weights:

$$\text{Overall Score} = 0.35 \times S_{\text{correctness}} + 0.25 \times S_{\text{safety}} + 0.20 \times S_{\text{robustness}} + 0.20 \times S_{\text{hallucination}} \quad (1)$$

A task passes if its overall score exceeds the weighted threshold (default 0.7).

5 Methodology

5.1 Benchmark Dataset

A benchmark dataset was created to evaluate code generation quality across diverse programming tasks:

Table 4: Benchmark Dataset Composition

Category	Tasks	%	Examples
Algorithms	10	37.0%	Factorial, Fibonacci, Binary Search, Quicksort, Merge Sort
Data Structures	5	18.5%	Stack, Queue, Linked List, Binary Tree, Hash Table
String Manipulation	5	18.5%	Reverse, Palindrome, Anagram, Word Count, Compression
File/API	4	14.8%	JSON Parse, CSV Parse, File I/O, HTTP Status
Error Handling	3	11.1%	Safe Divide, Email Validation, Integer Parsing
Total	**27**	**100%**	

Table 5: Difficulty Distribution

Difficulty	Count	Percentage
Easy	14	51.9%
Medium	10	37.0%
Hard	3	11.1%

Each task includes:

- Natural language requirements (`input_text`)
- Reference implementation (`reference_output`)
- Test cases with assertions (`metadata.test_cases`)
- Category and difficulty metadata

5.2 Experiment Configuration

Table 6: Experiment Configuration

Parameter	Value
Model	claude-sonnet-4-20250514
Temperature	0.7
Max Tokens	2048
Generations per Task	1 (single generation)
Correctness Threshold	0.7
Robustness Threshold	0.6
Safety Threshold	0.7
Hallucination Threshold	0.7

5.3 Experiment Procedure

The experiment followed this procedure:

1. Load benchmark dataset (27 tasks)
2. For each task:
   (a) Generate code using Claude with formatted prompt
   (b) Extract code from markdown response
   (c) Run evaluation pipeline (4 evaluators)
   (d) Record detailed results
3. Aggregate results and compute statistics
4. Generate visualizations

Checkpoints were saved every 5 tasks for reproducibility.

6 Testing and Validation

6.1 Unit Test Suite

A comprehensive unit test suite was developed to validate the evaluation framework components. The test suite ensures reliability of the core evaluation modules that assess LLM-generated code quality.

Table 7: Unit Test Coverage by Module

Module	Tests	Coverage	Lines	Missing
base.py	40	98%	104	2
correctness.py	45	100%	119	0
robustness.py	35	100%	154	0
safety.py	55	99%	177	2
hallucination.py	60	98%	208	5
metrics.py	52	100%	169	0
Total	**287**	**99%**	**938**	**9**

6.2 Test Categories

The test suite covers five primary categories ensuring comprehensive validation:

- **Initialization Tests**: Verify evaluator configuration, default parameters, and proper instantiation of evaluation components
- **Pattern Detection Tests**: Validate vulnerability pattern matching in safety evaluator and hallucination pattern detection for fake APIs
- **Scoring Tests**: Verify score calculation algorithms including weighted aggregation, threshold comparisons, and penalty application

- **Edge Case Tests**: Test boundary conditions, empty inputs, malformed code, and error handling paths
- **Integration Tests**: Verify end-to-end pipeline orchestration and inter-module communication

6.3 Testing Infrastructure

The testing infrastructure employs industry-standard tools and practices:

- **Framework**: pytest with pytest-cov for coverage reporting and pytest-asyncio for asynchronous test support
- **Fixtures**: 20+ shared fixtures in `conftest.py` providing mock data, sample code snippets, and evaluation contexts
- **Mocking**: `unittest.mock` for isolating external dependencies (Bandit static analyzer, subprocess calls, file system operations)
- **CI Integration**: GitHub Actions workflow executes the full test suite on every commit and pull request
- **Coverage Enforcement**: Minimum 95% coverage threshold enforced in CI pipeline

The high test coverage (99%) provides confidence in the evaluation framework's reliability and facilitates safe refactoring and extension of the codebase.

7 Results and Evaluation

7.1 Overall Performance

Table 8: Overall Experiment Results

Metric	Value
Total Tasks	27
Passed	25
Failed	2
Pass Rate	92.59%
Average Overall Score	0.905

The system demonstrates strong overall performance with 92.59% of tasks passing the evaluation threshold. The average overall score of 0.905 indicates high-quality code generation across diverse task types.

7.2 Per-Evaluator Results

Table 9: Per-Evaluator Performance Statistics

Evaluator	Avg Score	Min	Max	Pass Rate	Issues
Correctness	0.778	0.681	0.831	92.59%	4
Robustness	0.926	0.778	1.000	100%	18
Safety	0.994	0.850	1.000	100%	1
Hallucination	0.994	0.850	1.000	100%	1

Key observations:

- **Safety and Hallucination**: Near-perfect scores (99.4%) demonstrate Claude's ability to generate secure code without fabricated APIs
- **Robustness**: Strong performance (92.6%) with 100% pass rate, though issues related to error handling and null checking were identified
- **Correctness**: Lowest average score (77.8%) driven by test failures, indicating room for improvement in functional accuracy

7.3 Issue Analysis

Table 10: Issue Breakdown by Category

Category	Severity	Count
No error handling	Info	12
No null handling	Low	6
Test failures	Medium	4
Suspicious API	Medium	1
Dangerous imports	Medium	1
Total		**24**

Most issues (75%) are related to defensive programming practices (error handling, null handling) rather than critical security vulnerabilities or functional failures.

7.4 Visualizations

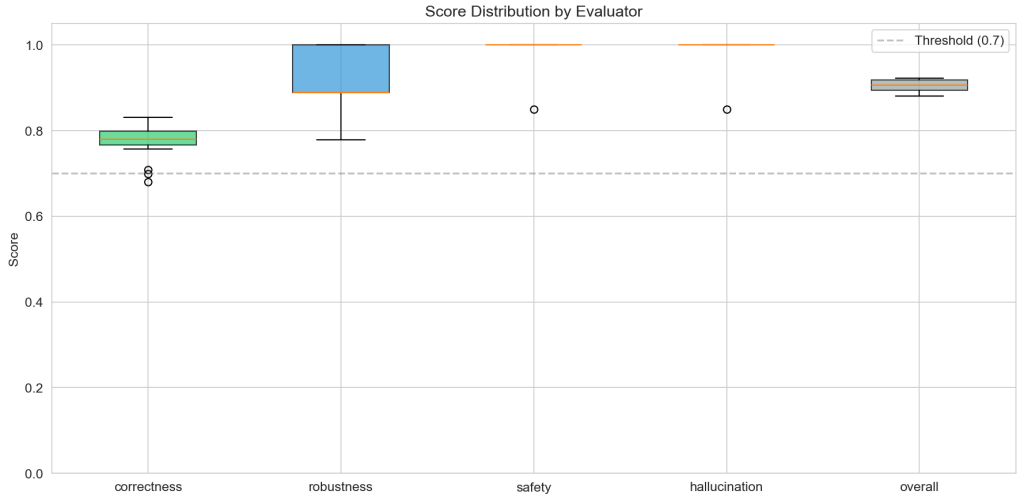


Figure 2: Score Distribution Across Evaluators. Box plots show the distribution of scores for each quality dimension. Safety and hallucination detection show consistently high scores with minimal variance, while correctness exhibits more variability.

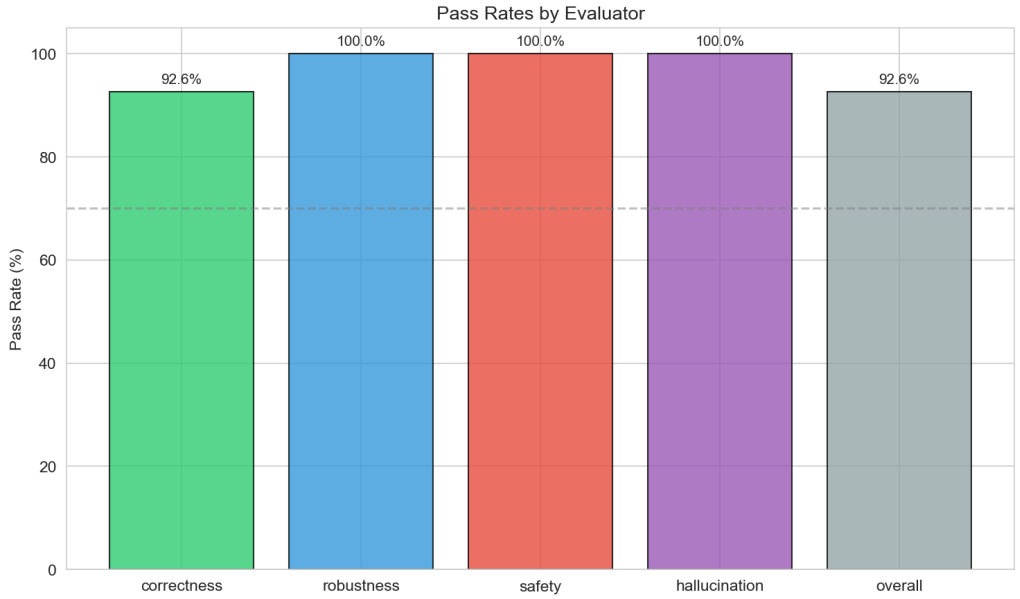


Figure 3: Pass Rates by Evaluator. All dimensions achieve pass rates above 90%, with safety, robustness, and hallucination detection achieving 100% pass rates.

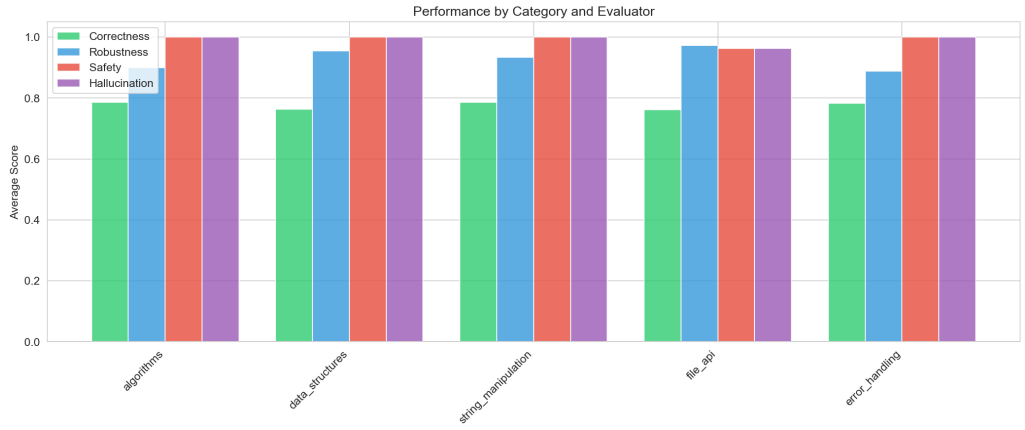


Figure 4: Performance by Task Category. Algorithm tasks show strong performance across all dimensions. Error handling tasks exhibit slightly lower correctness scores due to the complexity of edge case handling.

Figure 5: Scores by Difficulty Level. Performance remains relatively stable across difficulty levels, with hard tasks showing slightly more variance in correctness scores.

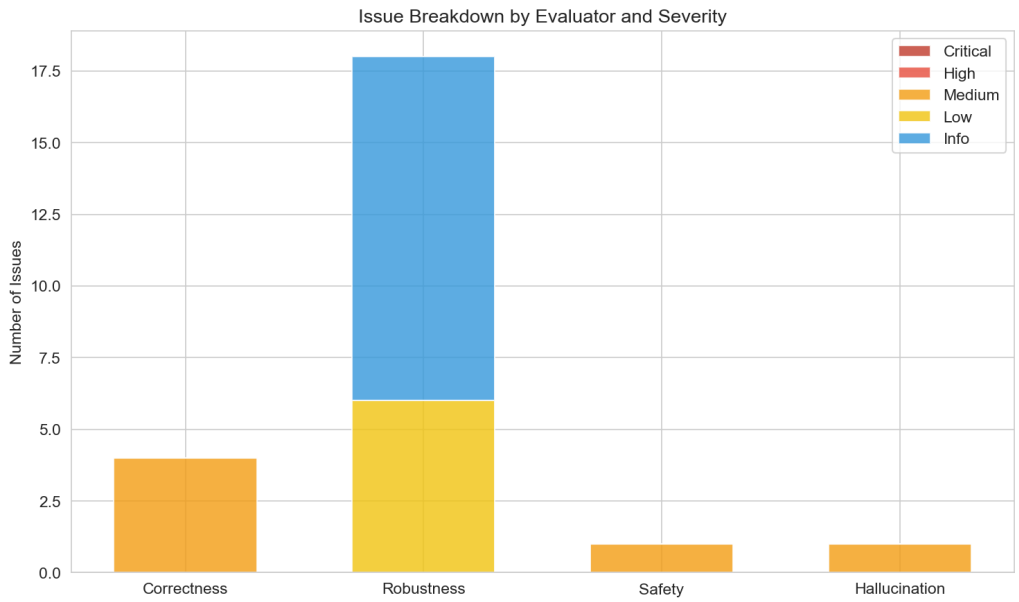


Figure 6: Issue Breakdown by Severity. The majority of issues are informational (no error handling), with only a small number of medium-severity issues related to test failures.

Figure 7: Correlation Between Evaluator Scores. Strong positive correlations exist between safety and hallucination detection, as both assess code authenticity. Correctness shows moderate correlation with other dimensions.

8 Discussion

8.1 Key Findings

1. **High Safety Scores**: Claude demonstrates excellent security awareness, avoiding common vulnerabilities like SQL injection and command injection. The 99.4% safety score indicates reliable generation of secure code.

2. **Minimal Hallucination**: The system rarely generates fabricated APIs or non-existent modules. This is critical for maintainable code that doesn't fail at runtime due to missing dependencies.

3. **Correctness Challenges**: While overall correctness is acceptable (77.8%), specific test failures indicate that LLM-generated code may not always handle edge cases as expected in test assertions.

4. **Defensive Programming Gaps**: The most common issues relate to missing error handling and null checks. These are not critical failures but represent opportunities for improvement.

8.2 Architecture Trade-offs

Table 11: Design Trade-offs

Aspect	Current Approach	Alternative
Evaluation	Automated static/dynamic analysis	Human expert review
Threshold	Fixed per-dimension (0.6-0.7)	Adaptive based on task complexity
Weights	Equal importance to safety	Task-specific weighting
RAG	Optional context augmentation	Mandatory for all generations

8.3 Strengths

1. **Modular Architecture**: Easy addition of new capabilities and evaluators

2. **Multi-Dimensional Evaluation**: Comprehensive assessment beyond simple correctness

3. **Production-Ready**: REST API, CLI, and React frontend for deployment

4. **Reproducibility**: Checkpoints, configuration logging, and deterministic evaluation

5. **High Test Coverage**: 287 unit tests achieving 99% code coverage across evaluation modules, ensuring reliability and enabling safe refactoring

8.4 Limitations

1. **Python-Only Evaluation**: Current evaluators optimized for Python; other languages have limited support
2. **Single Model**: Only Claude Sonnet tested; comparison with other models needed
3. **Dataset Size**: 27 tasks provide initial validation but larger benchmarks would strengthen conclusions
4. **No Pass@k**: Single generation per task; multiple samples could improve correctness

8.5 Lessons Learned

1. Prompt engineering significantly impacts code quality; structured prompts with examples yield better results
2. Temperature setting (0.7) balances creativity and consistency; lower values may improve correctness
3. Test case quality is critical for accurate correctness evaluation; edge cases should be comprehensive
4. Static analysis tools (Bandit) complement pattern matching for security assessment

9 Conclusion and Future Work

9.1 Summary of Contributions

This project successfully developed SE-Agent, a comprehensive AI-powered software engineering assistant with:

1. A modular agent architecture supporting five software engineering capabilities
2. A four-dimensional evaluation framework assessing correctness, robustness, safety, and hallucination
3. A benchmark dataset of 27 coding tasks across five categories and three difficulty levels
4. Empirical validation demonstrating 92.59% pass rate and 0.905 average score
5. Production-ready deployment with REST API, CLI, web interfaces, and 99% test coverage

The system demonstrates that LLM-generated code can achieve high safety (99.4%) and low hallucination rates (99.4%) while maintaining acceptable correctness (77.8%) and robustness (92.6%).

9.2 Future Work

Several directions merit further investigation:

1. **Multi-Language Support**: Extend evaluators to JavaScript, TypeScript, Java, and Go
2. **Pass@k Evaluation**: Generate multiple samples and select best to improve correctness
3. **Model Comparison**: Benchmark Claude against GPT-4, Code Llama, and specialized coding models
4. **Fine-Tuned Models**: Explore domain-specific fine-tuning for improved code generation
5. **Larger Benchmarks**: Expand dataset to 100+ tasks with more complex scenarios
6. **User Studies**: Evaluate system utility with professional software engineers
7. **Real-Time Evaluation**: Integrate evaluation into IDE plugins for continuous feedback

References

[1] Chen, M., Tworek, J., Jun, H., et al. (2021). Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.

[2] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.

[3] Yao, S., Zhao, J., Yu, D., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR)*.

[4] Anthropic. (2024). Claude Model Documentation. https://docs.anthropic.com/claude/docs

[5] Liu, J., Xia, C. S., Wang, Y., & Zhang, L. (2023). Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. *arXiv preprint arXiv:2305.01210*.

[6] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP-IJCNLP 2019*, 3982-3992.

[7] PyCQA. (2024). Bandit: Security Linter for Python. https://bandit.readthedocs.io/

[8] OWASP Foundation. (2024). OWASP Top Ten. https://owasp.org/www-project-top-ten/